

DOSSIER TECHNIQUE IRIS PROJECT

Flowers Lab



Flowers Lab
iris project

Joé Paday
Jérémy Indelicato

Table des matières

1.Introduction.....	2
2. Objectifs du Projet	3
3. Architecture du Pipeline.....	3
4. Étapes de Création du Projet.....	4
5. Outils Utilisés.....	4
6. Entraînement du Modèle et Résultats	5
7. Analyse Critique et Pistes d'Amélioration	5
8. Annexe	6

1.Introduction

Dans le cadre de notre projet iris-pipeline au sein du laboratoire FlowersLab, nous avons réalisé un projet complet de Data Pipeline. Ce projet s'inscrit dans une démarche professionnelle visant à simuler le travail de data scientist dans un environnement de production. Nous avons été amenés à concevoir un pipeline de données automatisé pour prédire la longueur des sépales de fleurs Iris à partir de leur largeur et d'autres caractéristiques pertinentes. Le projet a été réalisé en groupe et devait répondre à des exigences précises en matière de modularité, de reproductibilité et d'intégration continue à l'aide de technologies modernes comme Docker, PostgreSQL, FastAPI et MLflow.

2. Objectifs du Projet

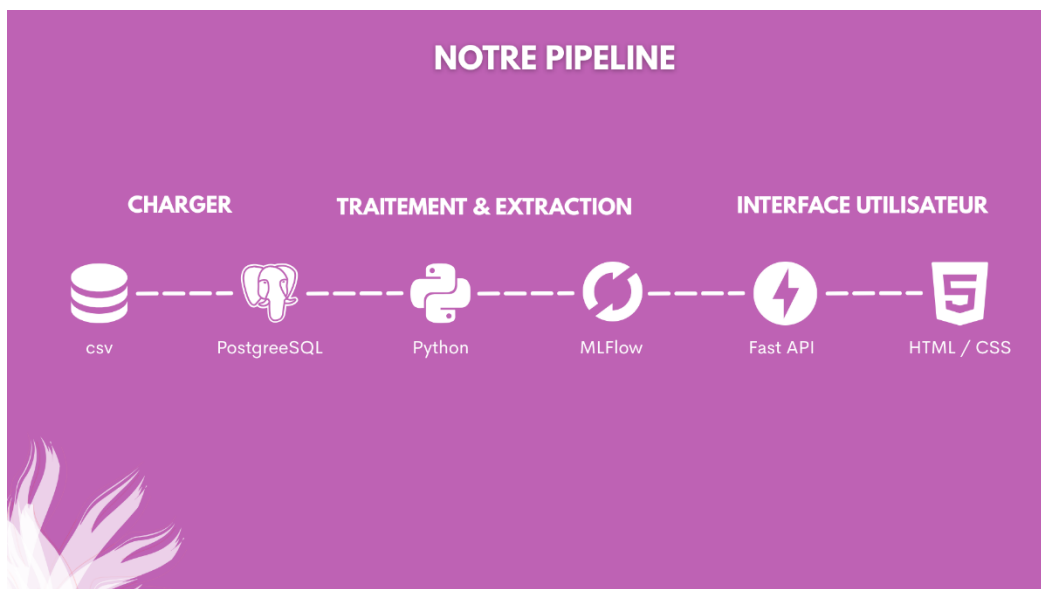
Le principal objectif de ce projet était de développer un pipeline de traitement de données capable d'automatiser l'ensemble du processus de prédiction à partir de données brutes. Le client fictif, un laboratoire botanique, souhaitait pouvoir prédire la longueur des sépales de fleurs Iris uniquement à partir de leur largeur et d'autres mesures.

Notre solution devait non seulement permettre l'entraînement d'un modèle de régression mais aussi exposer ce modèle via une API REST. Un des critères fondamentaux était que le tout devait être containerisé via Docker et orchestré avec Docker Compose pour garantir une portabilité et une maintenabilité exemplaires.

3. Architecture du Pipeline

L'architecture du pipeline repose sur quatre services distincts chacun dans un conteneur dédié :

- Un service de prétraitement qui nettoie et insère les données dans une base PostgreSQL.
- Un service de modélisation qui extrait les données, entraîne un modèle, puis le logue dans MLflow.
- Une base de données PostgreSQL contenant les données d'entraînement.
- Une API REST basée sur FastAPI qui permet à un utilisateur d'obtenir des prédictions.



Chaque service communique avec les autres via un réseau Docker partagé. Cette organisation modulaire permet de maintenir, tester et mettre à jour chaque composant indépendamment des autres, ce qui est essentiel dans un environnement professionnel.

4. Étapes de Création du Projet

La première étape a été le nettoyage du jeu de données `iris.csv`, en s'assurant que les noms de colonnes étaient cohérents et utilisables par les scripts Python. Ensuite, nous avons mis en place un script de prétraitement pour insérer ces données dans PostgreSQL. Après la validation de la base, nous avons conçu un modèle de machine learning à l'aide de `RandomForestRegressor` de Scikit-learn. Ce modèle a été évalué avec la métrique MSE, puis enregistré via MLflow. Nous avons ensuite développé une API REST en FastAPI exposant une route `/predict`. Enfin, tous les composants ont été intégrés et testés dans des conteneurs Docker, orchestrés avec Docker Compose.

5. Outils Utilisés

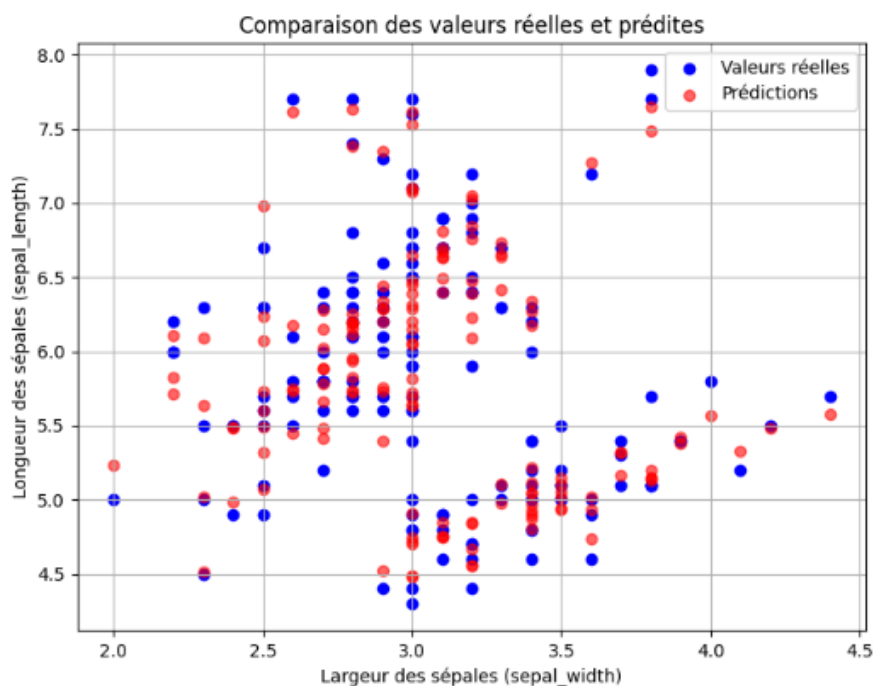
Plusieurs outils technologiques ont été mis en œuvre :

- Docker : pour containeriser chaque composant et assurer leur portabilité.
- Docker Compose : pour orchestrer tous les services en une seule commande.
- PostgreSQL : pour stocker les données structurées du dataset.
- Scikit-learn : pour l'entraînement du modèle de prédiction.
- FastAPI : pour exposer une API REST robuste et moderne.
- MLflow : pour tracker les expériences et enregistrer les modèles et métriques associées.

Ces outils ont permis d'implémenter un pipeline professionnel, automatisé et traçable de bout en bout.

6. Entraînement du Modèle et Résultats

Nous avons utilisé un `RandomForestRegressor`, entraîné sur 80% des données disponibles et testé sur les 20% restants. Le modèle a été logué dans MLflow avec une Mean Squared Error (MSE) inférieure à 0.1, ce qui montre une très bonne performance pour un problème de régression. Les prédictions se sont révélées précises et cohérentes avec les données réelles. Un graphique de visualisation comparant valeurs réelles et prédites a été généré et confirmé cette bonne performance.



7. Analyse Critique et Pistes d'Amélioration

Le projet a été mené avec succès et tous les objectifs ont été atteints.

Toutefois, quelques axes d'amélioration peuvent être envisagés :

- Intégration d'une interface utilisateur (bonus en cours de développement).
- Tests unitaires sur chaque composant pour garantir leur stabilité.
- Monitoring plus avancé des logs, des performances et des erreurs.
- Possibilité de versionner les datasets et pipelines pour faciliter l'évolution.

Le passage en production nécessiterait une configuration de sécurité plus poussée, notamment au niveau de l'authentification sur l'API et de l'accès à la base de données.

8. Annexe

Lien du Github : <https://github.com/ZyTyyy/iris-pipeline>

Lien du Trello : <https://urlr.me/e3uvmn>